VIDYA JYOTHI INSTITUTE OF TECHNOLOGY

AN AUTONOMOUS INSTITUTION

(Accredited by NAACA+ Approved by AICTE New Delhi & Permanently Affiliated to JNTUH)

Aziz Nagar Gate, C.B. Post, Hyderabad-500075

DEPARTMENT OF INFORMATION TECHNOLOGY

LAB PROJECT REPORT

Roll No	:	**24911A1283**
Name	:	**G kavya sri**
Project Title	:	**Hospital Management System**
Course Name	:	**Object Oriented Programming through Java Lab**
Course Code	:	**A224592**
Academic Year	:	**2025-26**
Regulations	:	**R22**
Year and Semester	:	**II B.Tech II Semester**

VIDYA JYOTHI
INSTITUTE OF TECHNOLOGY
AN AUTONOMOUS INSTITUTION

CERTIFICATE

This is to certify that the Object Oriented Programming through Java Lab project report entitled " **Hospital Management System** " submitted by **G kavyasri (24911A1283)** to Vidya Jyothi Institute of Technology (An Autonomous Institution), Hyderabad, of B. Tech.
II year, II Semester in Information Technology a Bonafide record of Lab project work carried out by them under my supervision.

Signature of Faculty
Mr.B Eswar Babu
Associate Professor

Signature of Head of the Department
Dr. A. Obulesu
Professor

Abstract

The Hospital Management System is a Java-based desktop application developed to simplify and automate the management of hospital operations. The system provides an efficient way to maintain patient information digitally, reducing manual paperwork and improving data organization. It is designed using Java Swing for the graphical user interface and JDBC for database connectivity with MySQL. The application enables hospitals to store and manage patient details securely and efficiently. The project includes important functionalities such as patient registration, doctor appointment booking, billing management, and medical record storage. The patient registration module allows users to add and manage patient details, while the appointment module helps in scheduling doctor consultations. The billing section generates and manages patient bills, and the medical record module stores patient health history for future reference. The system uses Object-Oriented Programming concepts like inheritance and collections to improve code reusability and data handling.

The Hospital Management System provides a user-friendly interface that makes hospital administration faster and more reliable. By integrating database management through JDBC and MySQL, the system ensures secure data storage and easy retrieval of information. This project helps improve hospital efficiency, reduces human errors, and enhances overall patient management. It can further be enhanced by adding modules such as online appointments, admin login, pharmacy management, and report generation.

Table of Contents

Chapter 1 – Introduction

1.1 Project Overview

The Hospital Management System is a Java-based desktop application developed to manage hospital operations digitally and efficiently. The project is designed using Java Swing for the graphical user interface and JDBC for connecting the application with a MySQL database. It helps in maintaining patient records, booking doctor appointments, managing billing information, and storing medical records in a structured manner.

1.2 Objectives

The main objectives of this project are:

- To develop a user-friendly hospital management application using Java.
- To automate patient registration and record management.
- To provide efficient doctor appointment booking functionality.
- To generate and manage billing details digitally.
- To store and retrieve medical records securely using MySQL database.

1.3 Scope of the Project

This system is designed for:

- The system can be used in hospitals, clinics, and healthcare centers.
- It helps in maintaining digital patient records efficiently.
- The project supports appointment scheduling and patient management.
- It can be extended to include online appointment booking features.
- The system can be enhanced with admin and doctor login modules.
- Future improvements may include pharmacy and laboratory management.

Chapter 2 – System Analysis

2.1 Functional Requirements

- Registernewpatients with ID, name, age, and disease.
- Book appointments for patients with doctor name and date.
- Generate billing details for patients.
- Store medical records of patients.
- Insert and store all data in MySQL database.
- Provide a GUI using Java Swing with different tabs.
- Allow user interaction for all hospital operations.

2.2 Non-Functional Requirements

- System should respond quickly to user actions.
- Data should be stored accurately in the database.
- Interface should be simple and user-friendly.
- System should be reliable and handle basic errors.
- Code should be easy to maintain and update..

2.3 Software Requirements

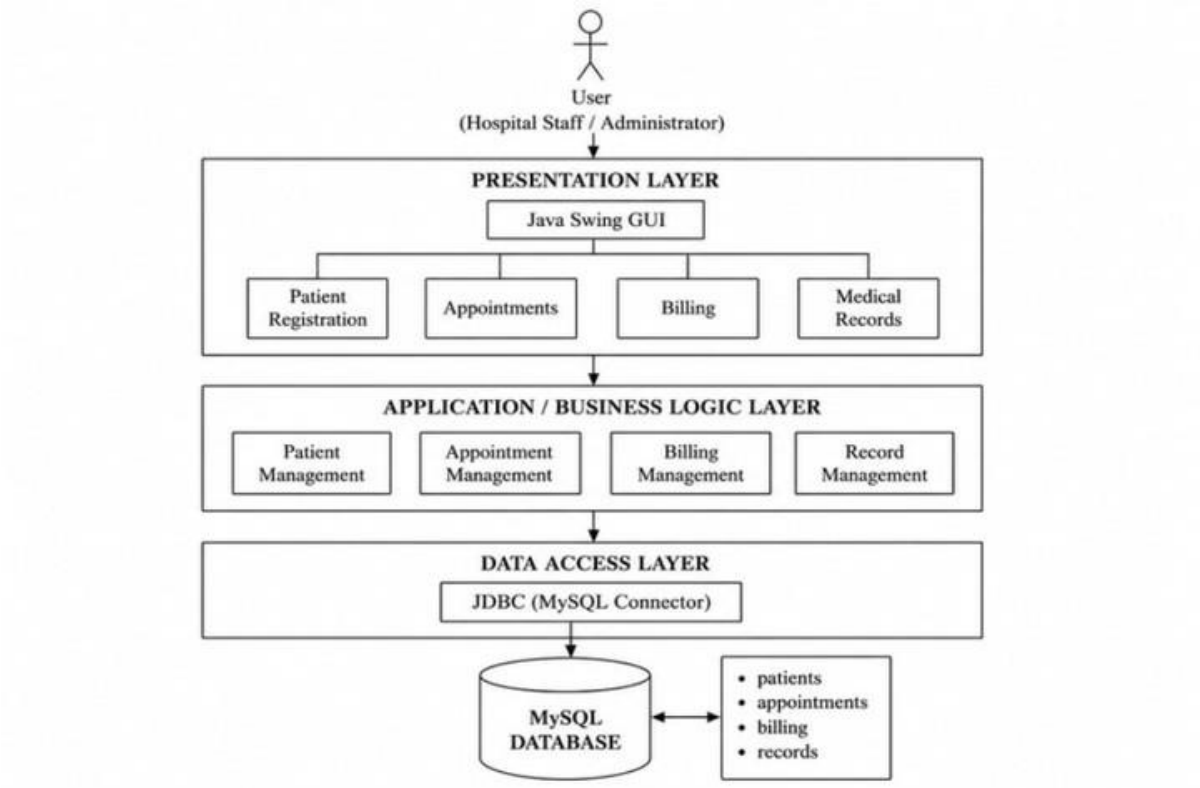
Requirement	Details
Operating System	Windows / Linux / macOS
Programming Language	Java (JDK 8 or above)
IDE	Eclipse / IntelliJ IDEA / NetBeans / VS Code
Database	MySQL
JDBC Driver	MySQL Connector/J
GUI Library	Java Swing (built-in in Java)

2.4 Hardware Requirements

Requirement	Details
Processor	Intel i3 or higher
RAM	Minimum 4 GB (8 GB recommended)
Storage	Minimum 500 MB free space (HDD/SSD)
Monitor	1366×768 resolution or higher
Input Devices	Keyboard and Mouse

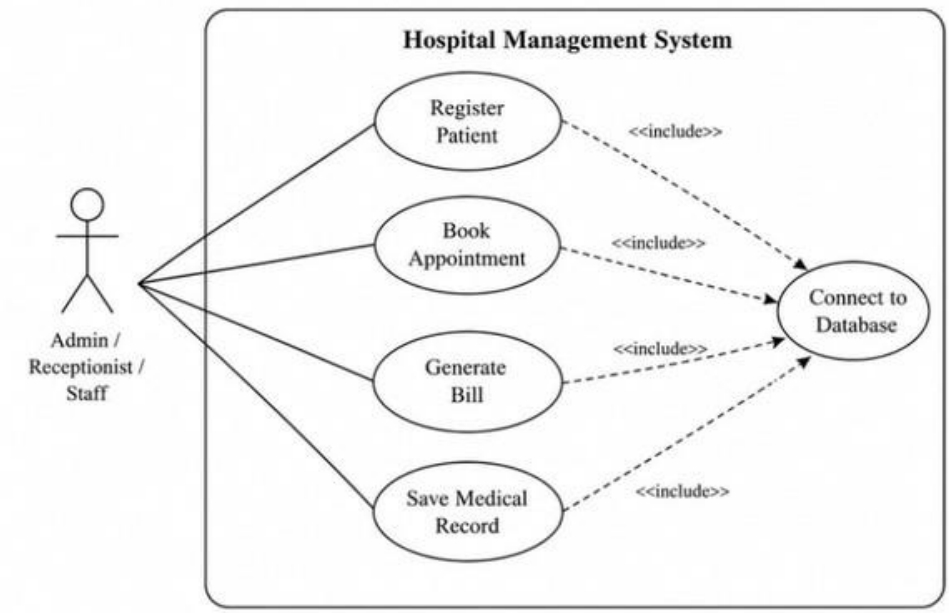
Chapter 3 – System Design

3.1 Architecture Diagram

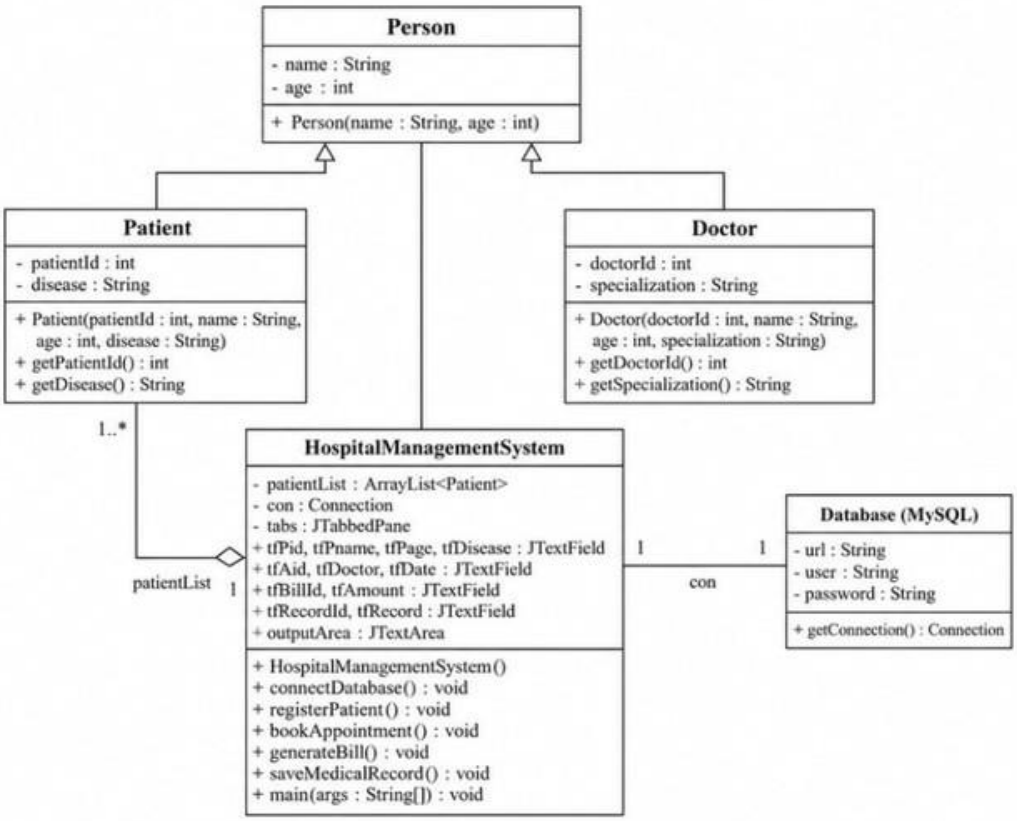


3.2 UML Diagrams

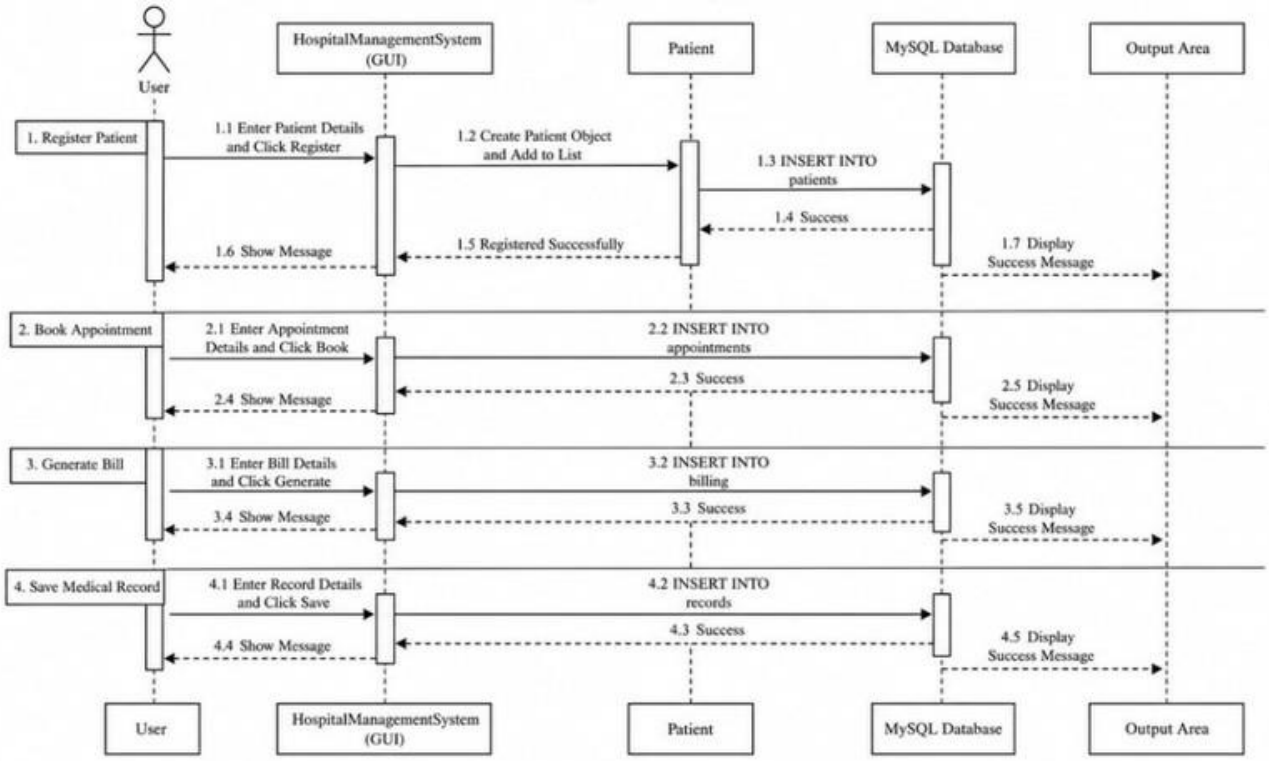
Use Case Diagram



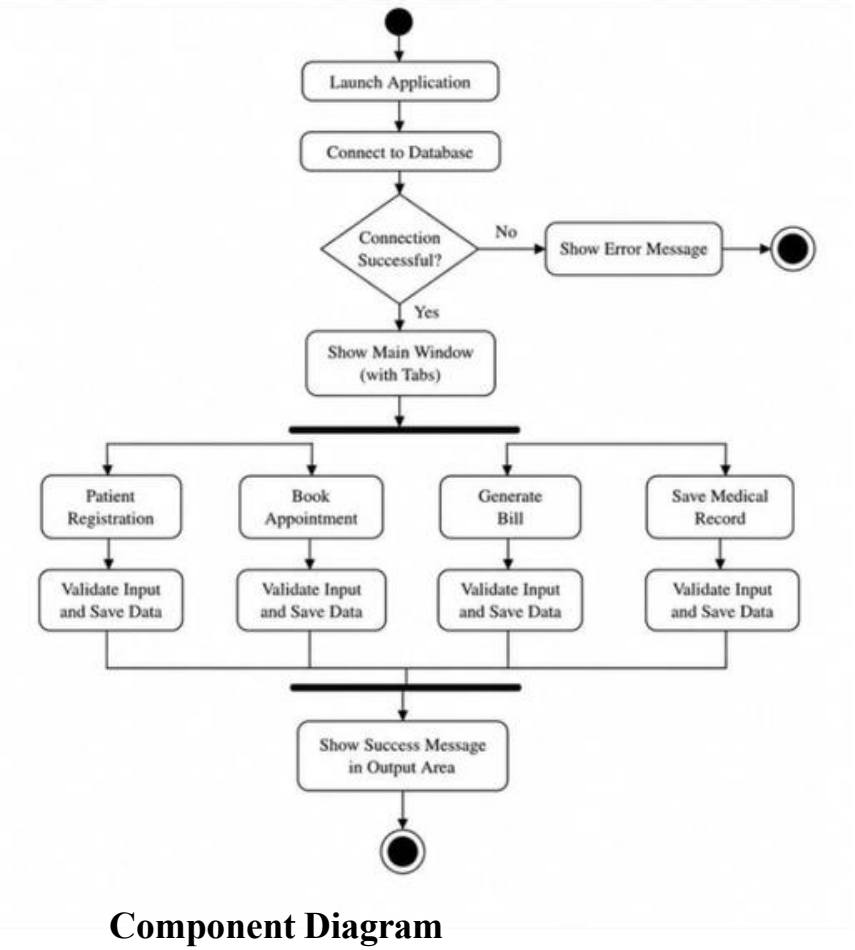
Class Diagram



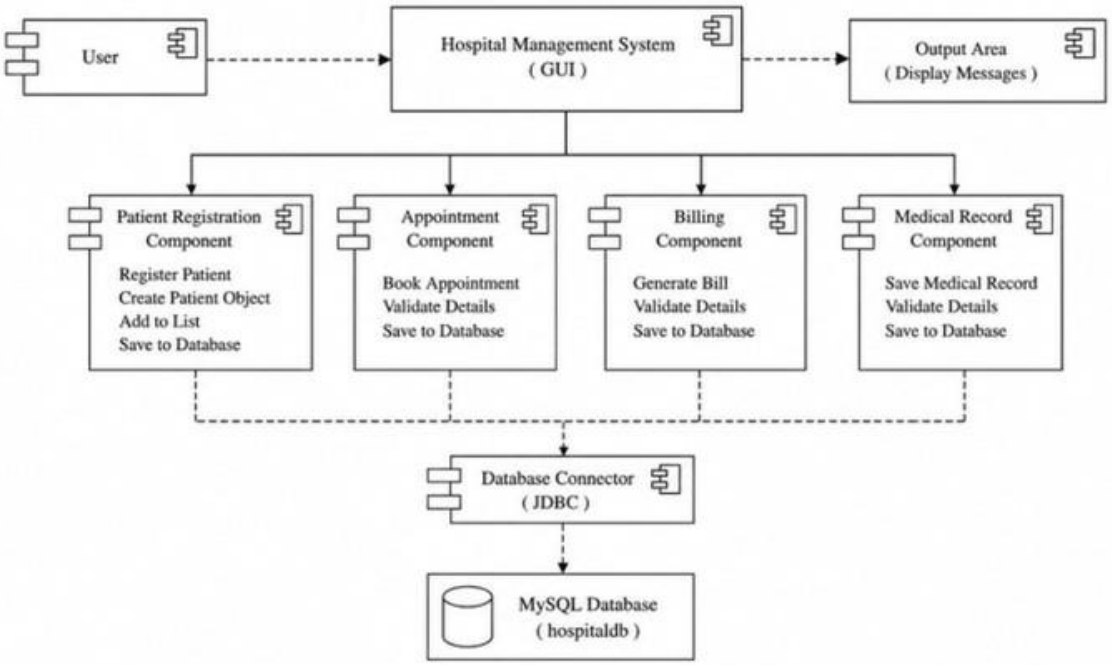
Sequence Diagram



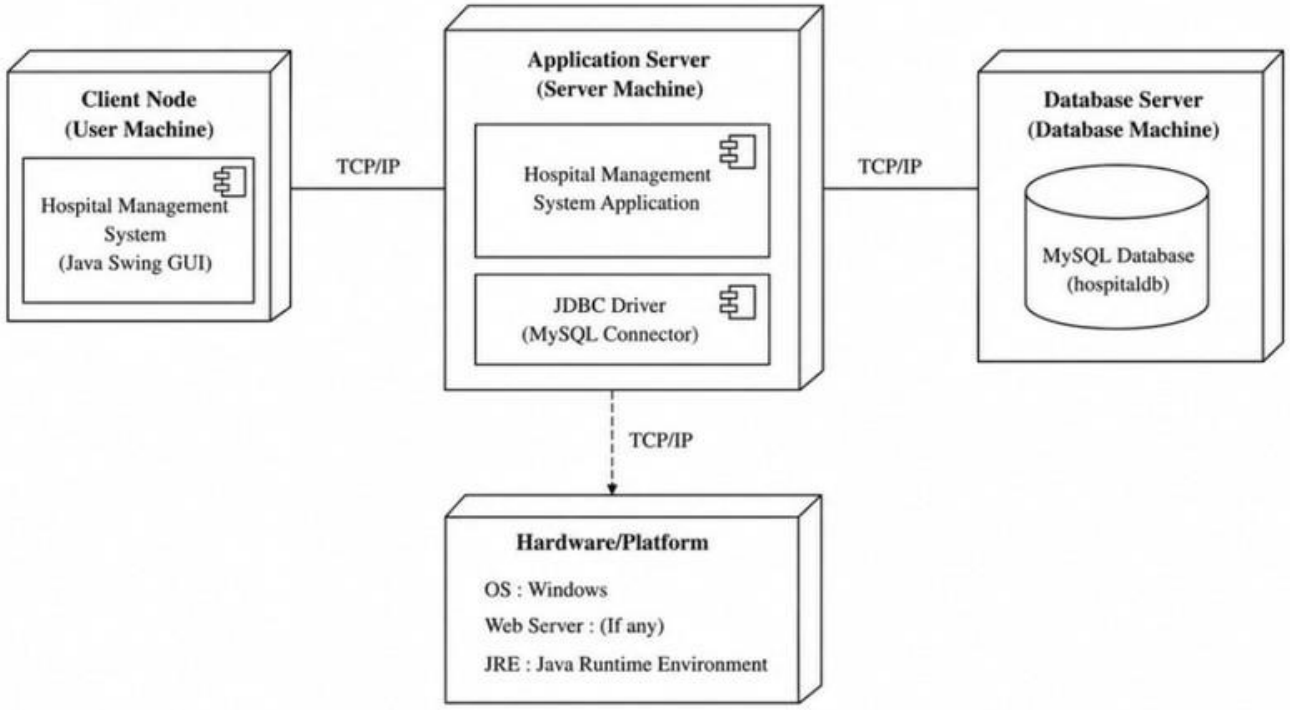
Activity Diagram



Component Diagram



Deployment Diagram



3.3 Database Design

Chapter 4 – Implementation

4.1 Modules Description

Module

Patient Registration Module

Appointment Module

Billing Module

Medical RecordsModule

Database Module

4.2 Important Java Concepts

- Classes and Objects
- Inheritance
- Polymorphism
- Exception Handling
- Collections (ArrayList for patient management)
- JDBC (Database Connectivity)
- AWT & Swing (GUI Design)
- Event Handling
- Multithreading (optional for multiple users)
- File Handling (optional record storage)

4.3 Code Snippets

Create Database

```
CREATEDATABASE hospitaldb;

USE hospitaldb;
```

Creating patient table

```
CREATE TABLE patients (
patient_id INT PRIMARY KEY,
name VARCHAR(50),
   age INT,
   disease VARCHAR(100)
);
```

Creating appointments table

```
(CREATE TABLE appointments
   patient_id INT,
 doctor_name VARCHAR(50),
 appointment_date VARCHAR(30)
);
```

Creating billing table

```
CREATE TABLE billing (
   patient_id INT,
 amount DOUBLE
);
```

Creating billing table

```
(CREATE TABLE records
   patient_id INT,
 medical_record VARCHAR(200)
);
```

DBConnection.java

```
 import javax.swing.;
import java.awt.;
import java.awt.event.;
import java.sql.;
import java.util.ArrayList;


// Parent Class
class Person {
protected String name;
protected int age;

public Per son(String name, int age) {
    this.nam e = name;
    this.age = age;
}

}

// Patient Class
class Patient extends Person {
private int patientId;
private String disease;

public Patient(int patientId, String name, int age, String disease) {
    super(name, age);
    this.patientId = patientId;
    this.disease = disease;
}

public int getPatientId() {
    return patientId;
}

public String getDisease() {
    return disease;
}

}
```

```
// Doctor Class
class Doctor extends Person {
private int doctorId;
private String specialization;


public Doctor(int doctorId, String name, int age, String specialization) {
    super(name, age);
    this.doctorId = doctorId;
    this.specialization = specialization;
}

public int getDoctorId() {
    return doctorId;
}

public String getSpecialization() {
    return specialization;
}
}
public class HospitalManagementSystem extends JFrame {

// Collections
ArrayList<Patient> patientList = new ArrayList<>();
//JDBC
Connection con;

// Tabs
JTabbedPane tabs;
// Patient Registration Fields
JTextField tfPid, tfPname, tfPage, tfDisease;
 // Appointment Fields
JTextField tfAid, tfDoctor, tfDate;
// Billing Fields
JTextField tfBillId, tfAmount;
// Medical Record Fields
JTextField tfRecordId, tfRecord;
JTextArea outputArea;
```

```
public HospitalManagementSystem() {
    setTitle("Hospital Management System");
    setSize(800, 600);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    tabs = new JTabbedPane();
 //================= PATIENT PANEL =================
    JPanel patientPanel = new JPanel();
    patientPanel.setLayout(new GridLayout(6, 2));
    tfPid = new JTextField();
    tfPname = new JTextField();
    tfPage = new JTextField();
    tfDisease = new JTextField();
    JButton addPatientBtn = new JButton("Register Patient");
    patientPanel.add(new JLabel("Patient ID"));
    patientPanel.add(tfPid);
    patientPanel.add(new JLabel("Patient Name"));
    patientPanel.add(tfPname);
    patientPanel.add(new JLabel("Age"));
    patientPanel.add(tfPage);
    patientPanel.add(new JLabel("Disease"));
    patientPanel.add(tfDisease);
    patientPanel.add(addPatientBtn);
    tabs.add("Patient Registration", patientPanel);
 //================= APPOINTMENT PANEL =================
    JPanel appointmentPanel = new JPanel();
    appointmentPanel.setLayout(new GridLayout(5, 2));
    tfAid = new JTextField();
    tfDoctor = new JTextField();
    tfDate = new JTextField();
    JButton bookBtn = new JButton("Book Appointment");
    appointmentPanel.add(new JLabel("Patient ID"));
    appointmentPanel.add(tfAid);
    appointmentPanel.add(new JLabel("Doctor Name"));
    appointmentPanel.add(tfDoctor);
    appointmentPanel.add(new JLabel("Appointment Date"));
   appointmentPanel.add(tfDate);
   appointmentPanel.add(bookBtn);
   tabs.add("Appointments", appointmentPanel);
```

```
// ================== BILLING PANEL ==================
    JPanel billingPanel = new JPanel();
    billingPanel.setLayout(new GridLayout(4, 2));
    tfBillId = new JTextField();
    tfAmount = new JTextField();
    JButton billBtn = new JButton("Generate Bill");
    billingPanel.add(new JLabel("Patient ID"));
    billingPanel.add(tfBillId);
    billingPanel.add(new JLabel("Amount"));
    billingPanel.add(tfAmount);
    billingPanel.add(billBtn);
    tabs.add("Billing", billingPanel);
   //================== RECORD PANEL ==================
    JPanel recordPanel = new JPanel();
    recordPanel.setLayout(new GridLayout(4, 2));
    tfRecordId = new JTextField();
    tfRecord = new JTextField();
    JButton saveRecordBtn = new JButton("Save Medical Record");
    recordPanel.add(new JLabel("Patient ID"));
    recordPanel.add(tfRecordId);
    recordPanel.add(new JLabel("Medical Record"));
    recordPanel.add(tfRecord);
    recordPanel.add(saveRecordBtn);
    tabs.add("Medical Records", recordPanel);
   //================== OUTPUT AREA ==================
    outputArea = new JTextArea();
    JScrollPane scroll = new JScrollPane(outputArea);
    add(tabs, BorderLayout.CENTER);
    add(scroll, BorderLayout.SOUTH);
   //================== DATABASE ==================
    connectDatabase();
//================== BUTTON ACTIONS ==================
       addPatientBtn.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            registerPatient();
        }
    });
```

```
bookBtn.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            bookAppointment();
        }
    });
   billBtn.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            generateBill();
        }
    });
   saveRecordBtn.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            saveMedicalRecord();
        }
    });
   setVisible(true);
}

// ================== DATABASE CONNECTION ==================
public void connectDatabase() {
    try {

        Class.forName("com.mysql.cj.jdbc.Driver");

        con=DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/hospitaldb",
                "root",
                "password");

        outputArea.append("Database Connected\n");

    } catch (Exception e){
        e.printStackTrace();
    }
}
```

```
// ================= PATIENT REGISTRATION ================
public void registerPatient() {
    try {
       int id = Integer.parseInt(tfPid.getText());
        String name = tfPname.getText();
        int age = Integer.parseInt(tfPage.getText());
        String disease = tfDisease.getText();
        Patient p = new Patient(id, name, age, disease);
        patientList.add(p);
        String query = "INSERT INTO patients VALUES (?, ?, ?, ?)";
       PreparedStatement pst = con.prepareStatement(query);
       pst.setInt(1, id);
        pst.setString(2, name);
        pst.setInt(3, age);
        pst.setString(4, disease);
        pst.executeUpdate();
        outputArea.append("Patient Registered Successfully\n");
    } catch (Exception e) {
        e.printStackTrace();
    }
}
// ================= APPOINTMENT ================
public void bookAppointment() {
   try {
        int pid = Integer.parseInt(tfAid.getText());
        String doctor = tfDoctor.getText();
        String date = tfDate.getText();
        String query = "INSERT INTO appointments VALUES (?, ?, ?)";
        PreparedStatement pst = con.prepareStatement(query);
        pst.setInt(1, pid);
        pst.setString(2, doctor);
        pst.setString(3, date);
        pst.executeUpdate();
        outputArea.append("Appointment Booked Successfully\n");
     } catch (Exception e) {
         e.printStackTrace();
    }
}
```

```
// ================== BILLING ==================
public voidgenerateBill(){

   try {
       int pid = Integer.parseInt(tfBillId.getText());
       double amount = Double.parseDouble(tfAmount.getText());
       String query = "INSERT INTO billing VALUES (?, ?)";
       PreparedStatement pst = con.prepareStatement(query);
       pst.setInt(1, pid);
       pst.setDouble(2, amount);
       pst.executeUpdate();
       outputArea.append("Bill Generated Successfully\n");
  } catch (Exception e) {
       e.printStackTrace();
    }
}
// ================= MEDICAL RECORD ================

    try {
       int pid = Integer.parseInt(tfRecordId.getText());
       String record = tfRecord.getText();
       String query = "INSERT INTO records VALUES (?, ?)";
       PreparedStatement pst = con.prepareStatement(query);
       pst.setInt(1, pid);
       pst.setString(2, record);
       pst.executeUpdate();
       outputArea.append("Medical Record Saved Successfully\n");
 } catch (Exception e) {
       e.printStackTrace();
    }
}
public void saveMedicalRecord(){
// ================= MAIN ================

public static void main(String[] args) {

    new HospitalManagementSystem();
}

}
```

Chapter 5 – Testing

5.1 Test Cases

5.2 Error Handling

The system handles the following exceptions:

- NumberFormatException (invalid numeric input)
- SQLException (database errors)
- NullPointerException (missing values)
- ClassNotFoundException (JDBC driver issues)
- InterruptedException (multithreading issues – optional)

Chapter 6 - Results and Screenshots

FIGURE 1:Entering patient details

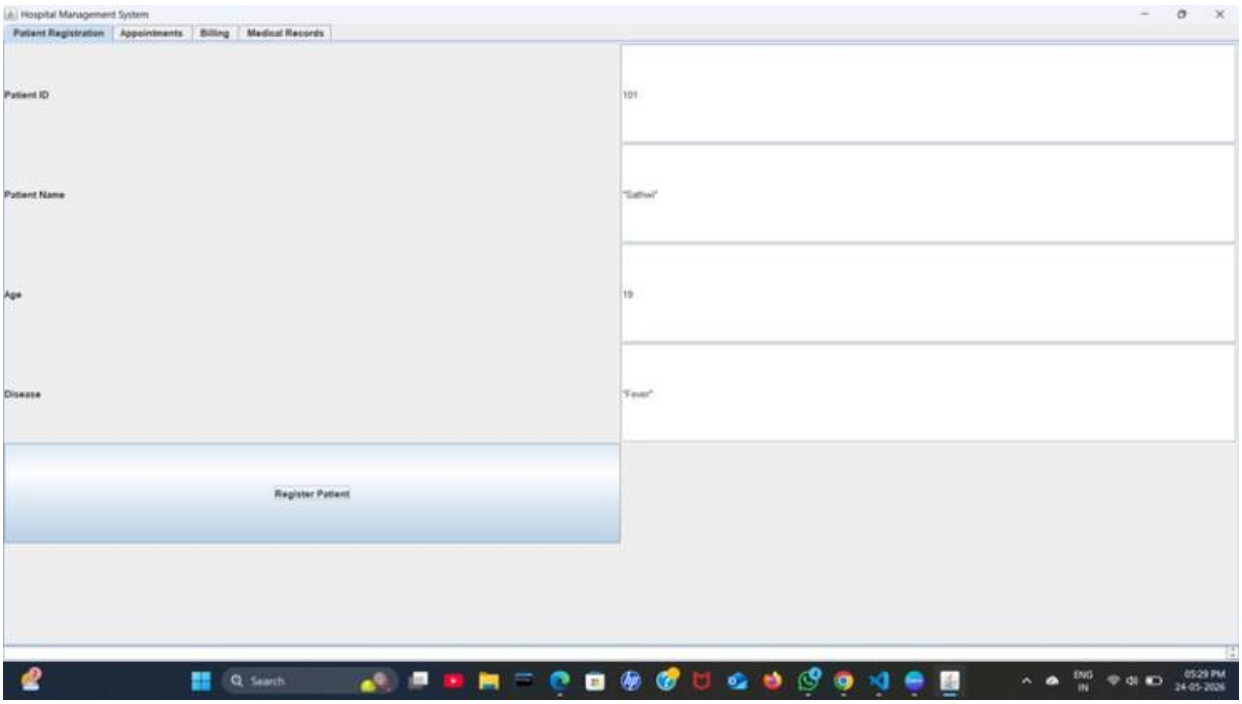


FIGURE 2 : Entering appointment details

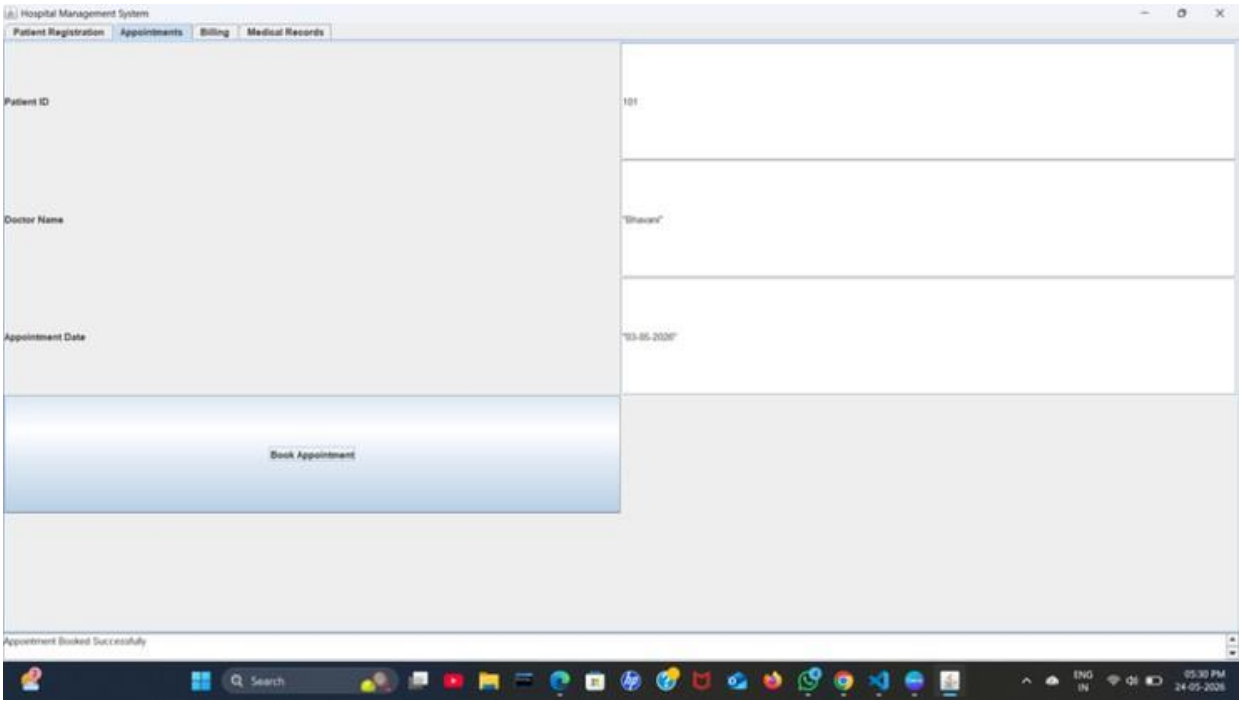


FIGURE 3: Entering billing details

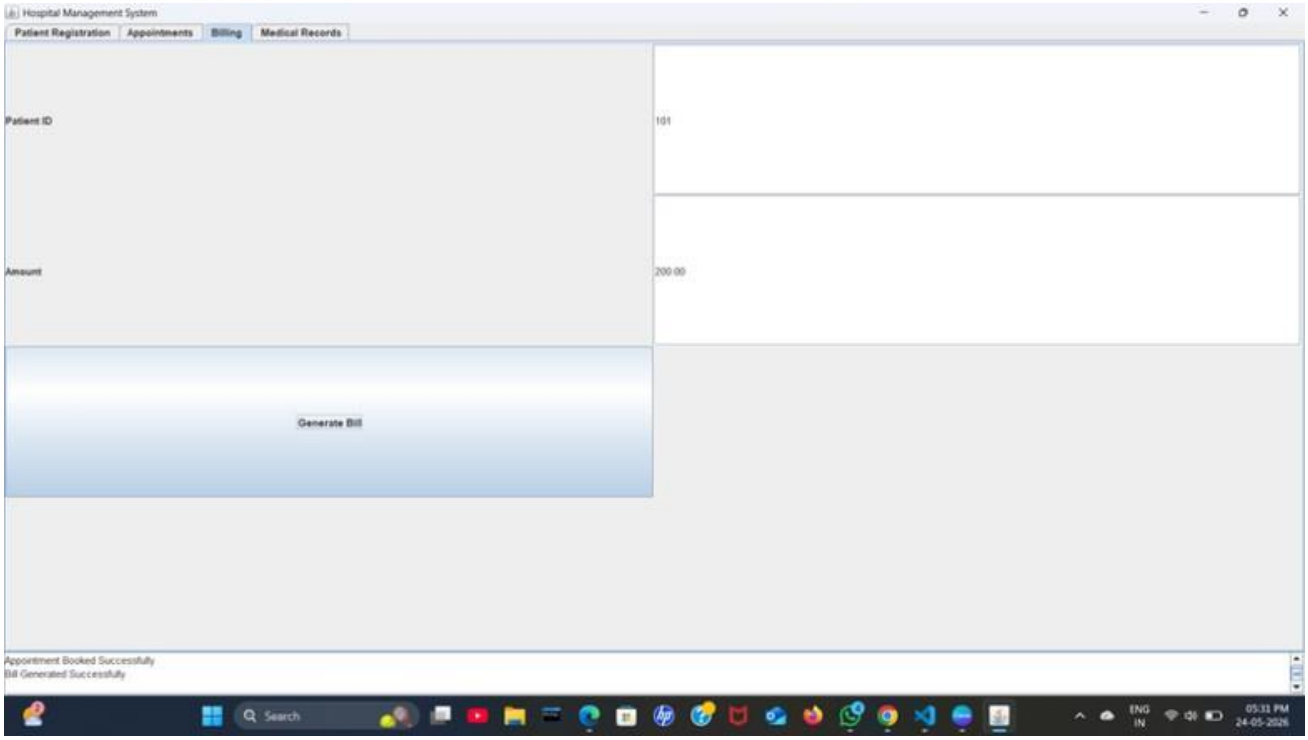


FIGURE 4 : Entering medical record details

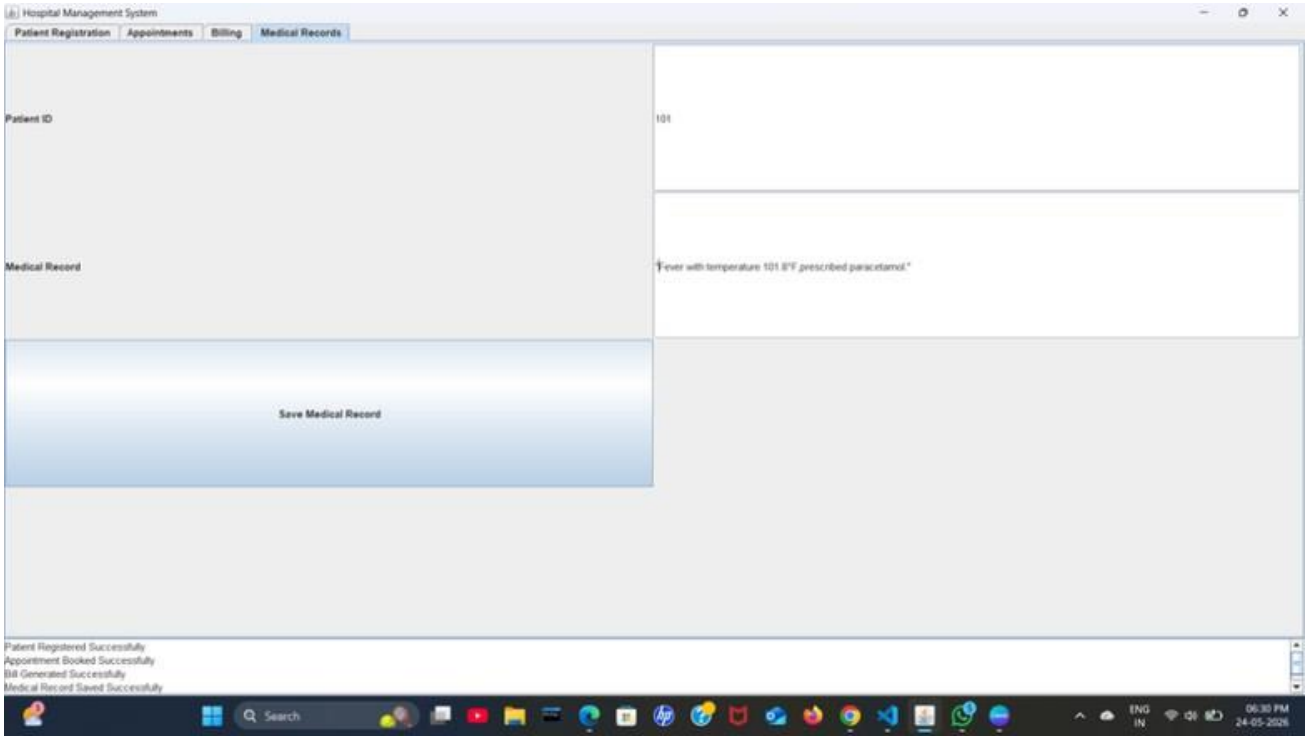


FIGURE 5: Displaying patient detials

```
mysql> use hospital_db;
Database changed
mysql> select *from  Patients;
+------------+------+------+----------+
| Patient_ID | Name | Age  | Disease |
+------------+------+------+----------+
|          1 | Ravi |   25 | Fever    |
+------------+------+------+----------+
1 row in set (0.01 sec)
```

FIGURE 6 : Displaying appointment details

```
ysql> SELECT * FROM Appointments;
+------------+-------------+------------------+
| Patient_ID | Doctor_Name | Appointment_Date |
+------------+-------------+------------------+
|          1 | Dr. Kumar   | 2026-06-10       |
+------------+-------------+------------------+
row in set (0.00 sec)
```

FIGURE 7: Displaying billing details

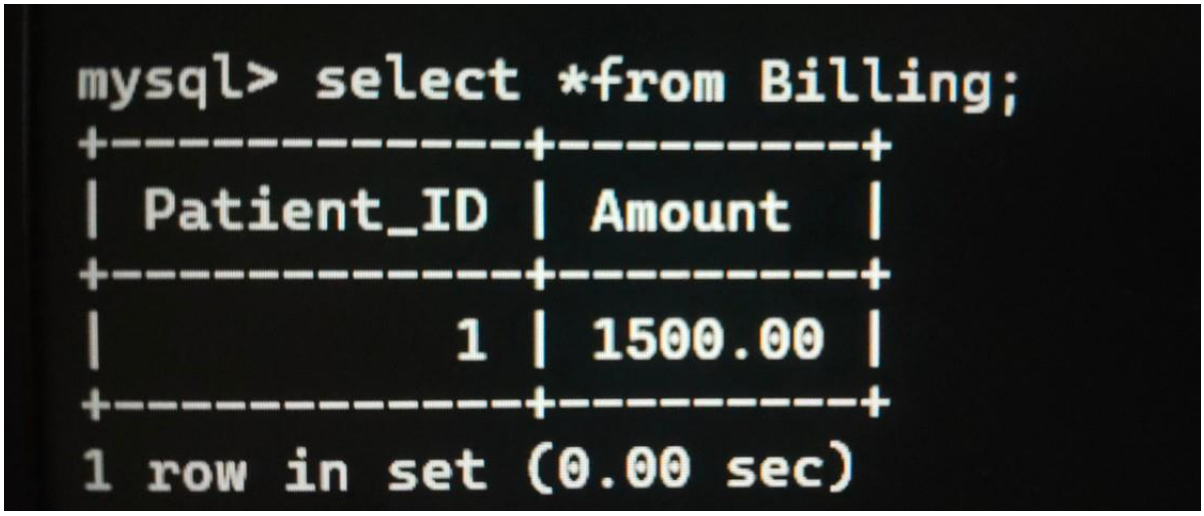
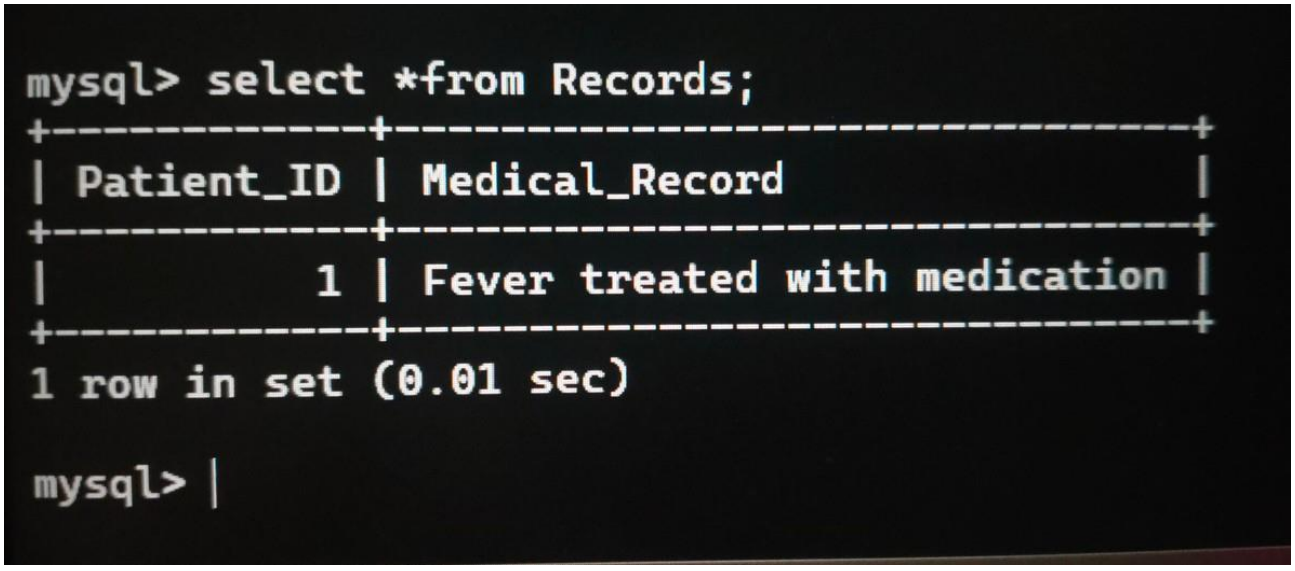


FIGURE 8 : Displaying medical records details



Chapter 7 – Conclusion and Future Scope

Conclusion

The Hospital Management System project was successfully developed using Java Swing and MySQL database connectivity. The system helps in managing important hospital activities such as patient registration, appointment booking, billing, and medical record maintenance.

The project uses Object-Oriented Programming concepts, JDBC, collections, and event handling to provide an efficient and user-friendly application. This system reduces manual work, improves data management, and increases the accuracy and speed of hospital operations.

Future Enhancements

- Add Doctor Management Module
- Implement Online Appointment Booking
- Add Patient Login and Admin Login Authentication
- Generate Printable Medical Reports and Bills
- Add Prescription Management System
- Implement Cloud Database Storage
- Add Email and SMS Notifications for Appointments

References

1. Java Programming by Herbert Schildt
2. Oracle Java Documentation
3. MySQL Official Documentation
4. JDBC API Documentation
5. Java Swing Tutorials
6. Eclipse / NetBeans IDE Documentation
7. W3Schools Java Tutorials
8. GeeksforGeeks Java and JDBC Articles
9. Stack Overflow Programming Discussions
10. Database System Concepts by Korth